



← Back to student guides

ClearML

Experiment tracking

3 levels

116 sections



The Complete Guide

Taught at three levels, written three ways. Pick Beginner, Mid-level, or Senior, then read the full explanation, a fast interview review, or the practical tips and traps for that level.

TASK

QUEUE

AGENT

MODEL

Part three of three, and the one to read if you only read one. A cumulative review of **the entire series** (foundations, platform machinery, and the governance and scale work a senior owns) organised by topic rather than by level. About fifty minutes. Fast review first, common questions at the end.





ClearML tracks experiments, versions datasets, and re-executes runs. `Task.init` records the code, commit, uncommitted diff, packages, arguments, metrics, and models. Because the environment is captured, any run can be cloned, reconfigured in the UI, and re-launched on an agent. The log is executable, not just readable.

Five components: SDK in your process; API server (MongoDB for metadata, Elasticsearch for metrics and logs, Redis for ephemeral state); web server; file server; agents pulling named queues.

The rules that never change

`Task.init` before every framework import, because capture is import-time patching. Read parameters from `connect` 's return value. It is bidirectional and overrides on an agent. One title , several series for one comparable chart; `report_single_value` for sortable columns. `output_uri` to object storage for anything large. `finalize()` is what makes a dataset version mean anything. Clone, do not rerun, since a completed task is a record.

Statuses and the loop

Status	Means
draft	Created or cloned; editable
queued	Waiting for an agent
in_progress	Running
completed / failed / aborted	Exit 0 / non-zero / stopped
published	Read-only and protected



the script, then exits locally.

The traps

Symptom	Cause	Fix
No scalars	<code>Task.init</code> after the framework import	Move it up
Stuck in <code>queued</code>	No agent on that queue	Start one; check the name
Git error on the agent	Commit never pushed	Push the branch
<code>ImportError</code> only remotely	Lazy import missed by detection	Module-level import, or declare it
Parameter edit ignored	Code reads its literal	Read <code>connect</code> 's return
File server full	Default <code>output_uri</code>	Point it at S3

Part two: platform machinery

Three entry points

Call	Patches	Captures env	Runs now	For
<code>Task.init</code>	Yes	Yes	Yes	Your training script
<code>Task.create</code>	No	No: declared	No	Launching code you do not modify



<code>task.clone</code>	NO	inherited	NO	returns with new parameters
-------------------------	----	-----------	----	-----------------------------

Reuse: `init` may reuse the last `empty` task from the same script: anti-clutter, wrong during a sweep, so `reuse_last_task_id=False`. Continuation: `continue_last_task=True` appends from the last iteration; right for resuming an OOM kill, wrong for a fresh run. Distributed: one task, init on rank 0, `Task.current_task()` elsewhere.

Capture and requirements

Bindings patch TensorBoard `add_*`, `torch.save`, `Model.save`, `joblib.dump`, XGBoost/LightGBM saves, matplotlib, `argparse`, and `hydra`. Requirements come from inspecting `sys.modules` at the end of the run, which produces three failures: lazy imports missed, CUDA wheels resolved as CPU builds, private packages unresolvable. Fixes: `set_packages`, `add_requirements`, and docker mode.

PYTHON

Copy

```
task.set_packages("requirements.lock")
task.set_base_docker("registry/train@sha256:...", docker_arguments="--shm-size=8g")
```

Agents, caches, queues

Cache	Holds
<code>venvs_cache</code>	Whole prebuilt virtualenvs, keyed on the requirement set
<code>pip_download_cache</code>	Wheels
<code>vcs_cache</code>	Bare git clones
<code>sdk.storage.cache</code>	Datasets, artifacts, models



never go on a GPU queue. a controller mostly sleeps and would hold the GPU while its own steps queue behind it. A worker listing `--queue urgent gpu` drains in order, which is a priority system for free.

HPO

PYTHON

Copy

```
HyperParameterOptimizer(  
    base_task_id=BASE, hyper_parameters=[...],  
    objective_metric_title="accuracy", objective_metric_series="val",  
    objective_metric_sign="max", optimizer_class=OptimizerOptuna,  
    execution_queue="gpu", max_number_of_concurrent_tasks=WORKERS,  
    total_max_jobs=40, min_iteration_per_job=5, max_iteration_per_job=30,  
    time_limit_per_job=45.0, save_top_k_tasks_only=10,  
)
```

A mismatched objective is silent and total: no ranking, no pruning, random search with a full budget spent. The base task must itself be a working completed run, or all N trials fail identically. `save_top_k_tasks_only` is what stops a sweep becoming a storage incident.

Datasets and pipelines

Concept	Say this
Storage shape	Chunked zips plus a file list; children store only their delta
<code>add_external_files</code>	Register bucket paths, no copy: immutability now depends on your bucket
<code>squash</code>	Collapse a long parent chain into one version for fetch speed
Cache placement	On ephemeral runners, a persistent volume is the whole startup cost
Pipeline caching	Keys on base task, parameters, and code: not input file contents
The fix	Version the data, so a change is a parameter change and thus a cache miss



References	<code>\${pipeline.x}</code> , <code>\${step.artifacts.n.url}</code> , <code>\${step.models.output.-1.url}</code>
Triggers	<code>TriggerScheduler</code> on a dataset or model tag; <code>TaskScheduler</code> for cron

Registry to serving



1 Train

Weights saved; model tagged `candidate` , linked to task and commit.



2 Evaluate

Metrics written into model metadata. This is what the gate reads.



3 Gate

A task compares against production with an explicit margin, and logs it.



4 Promote

Tags move; the winner is published and becomes read-only.



5 Serve

`auto-update` follows the tag, so the tag move `is` the deploy.

Tag vocabulary must be agreed and set from code only: `prod` vs `production` means serving silently keeps the old model. `clearml-session --base-task-id` gives a shell in the exact failing environment, and holds a worker until `--shutdown` .

Part three: trust, scale, and ownership

The trust model



Actor	Needs	Must not have
Researcher	Read all; enqueue to <code>cpu</code> / <code>gpu</code>	Enqueue to production or serving queues
CI on a protected branch	Enqueue to <code>ci</code>	Production cloud credentials
CI on a fork PR	Nothing, or an isolated ephemeral queue	Any workspace credential
Agent host	Data read, artifact write	Delete on the bucket
Serving	Registry read	Any write
Retention job	Scoped, audited delete	Write or enqueue

Three sharpening properties: **credentials are workspace-wide by default** (a leaked researcher key reads everything and can enqueue); **the uncommitted diff is stored on the server** (so an untracked `.env` in your working tree is now in the task record and in backups); and **agents hold ambient cloud credentials** that any task landing on them inherits.

The most dangerous configuration

A public repo, a `pull_request` workflow with workspace credentials, enqueueing to a queue whose agents hold production cloud credentials. Any stranger opening a PR gets code execution with those credentials. Fork builds go to ephemeral, credential-free agents, or behind explicit approval.

Queues as a security boundary

TEXT

Copy

```
services      ← platform team + schedulers.  Creds: registry read/write
gpu / cpu     ← researchers.                  Creds: data read, artifact write
ci            ← protected branches.           Creds: artifact write only
```



Credentials: one per purpose, so revocation is surgical and the audit log names a caller.

`CLEARML_WORKER_ID` set explicitly. Workload identity (IRSA, GKE workload identity, instance profiles) for cloud access, so there is nothing on disk to rotate or leak.

Self-hosting

Component	Holds	If it dies	Grows with
MongoDB	Tasks, models, users, queues	Everything stops: system of record	Task count
Elasticsearch	Scalars, plots, console logs	Metrics unavailable; tasks still run	Reporting rate × runs
Redis	Locks, worker state	Workers misbehave; usually self-heals	Fixed
API / web server	Stateless	Restart or scale out	-
File server	Artifacts	Artifacts unreadable: replace with S3	Artifact volume

Elasticsearch breaks first, and the symptom is silent

Metrics and logs grow and never shrink. Cross the high disk watermark and indices go read-only, so **experiments stop reporting while training continues**. Elastic disk is your primary platform alert, and you should know the read-only-release command before you need it.

Backups and upgrades

Three parts: `mongodump` of `backend` + `auth` ; an **Elasticsearch snapshot**; and the artifact bucket with versioning. A Mongo-only backup restores a complete-looking system with empty Scalars tabs. That is the mistake. Verify a restore in order: login → a known task's scalars →



Multi-tenancy

The open-source server is **one workspace with no RBAC**. Everyone with credentials can do everything. Say that plainly. What you can do: nested projects by convention, separate queues with fixed workers as the quota, **separate agent pools with separate credentials as the real isolation**, bucket prefixes per team with their own lifecycle, mandatory team and cost-centre tags. Hard isolation means a server per tenant. A twenty-line internal wrapper turns every one of those conventions into a default.

Cost

Centre	Driver	Control
GPU-hours	Trials × epochs × idle	HPO budgets, autoscaling, session timeouts, utilisation reporting
Artifact storage	Checkpoints × runs × retention	Best + last only, lifecycle rules, save_top_k_tasks_only
Dataset storage	Versions × copies	External files, squashing
Server infra	Task count and reporting rate	Retention, reporting discipline
Egress	Cross-region movement	Co-locate compute and storage

Idle GPU is nearly always the largest line: abandoned sessions, runs that fail late, sweeps with no pruning, workers idle between bursts. Spot instances need **retry_on_failure** plus checkpointing and **continue_last_task** , or you have bought cheaper failures.

Scale, in order of what breaks



1

Reporting rate



2 Task count

MongoDB queries slow; the table and search get sluggish. Fix: retention on CI and sweep tasks.

3 Artifact volume

Cost, and the file server's single disk. Fix: S3 plus lifecycle rules.

4 Worker count

API server CPU under hundreds of pollers. Fix: scale replicas. It is stateless.

Retention must be a scheduled task, dry-run first, exempting published models, **keep** -tagged tasks, and anything a registered model still references. Deleting a task can delete a production model's weights.

Reproducibility

Layer	Recorded	Pin with
Code	Yes: commit + diff	Nothing more
Packages	Detected names/versions	A hash-pinned lock file
Python	Recorded	The base image
CUDA, system libs	No	The base image digest , not a tag
Data	Only via a Dataset	Version + alias
Seeds	Only if connected	A connected parameter

"We are reproducible" is a claim until a weekly rebuild check clones a past release, reruns it, and compares the metric within a tolerance. The first run will fail; that is the point.

Lineage and erasure



promotion task's user, timestamp, comparison, and threshold). Promotion must be a task, never a manual tag move, or there is no approval record.

Erasure against an immutable content-addressed store touches dataset chunks, child versions, task artifacts, **debug samples**, console logs in Elasticsearch, backups, and weights, which you cannot meaningfully erase. The architectural fix is upstream: version derived or pseudonymised data, keep the identity mapping in a system built for deletes, and crypto-shred where raw data must be versioned.

Where ClearML stops

Need	Better tool
Point-in-time feature serving	A feature store
SQL analytics over versioned tables	Iceberg / Delta / a warehouse
Company-wide orchestration	Airflow, Dagster, Temporal
High-scale low-latency serving	KServe, Triton directly, a managed endpoint
Data quality contracts	Great Expectations, dbt tests
Fine-grained RBAC	The paid tier, or a server per tenant

ClearML is strongest as the experiment-to-model spine; weakest as a general orchestrator, a feature store, and a serving mesh.

Common interview questions

01

Design ClearML for a team of thirty across three squads.



worker counts as quota, `ci` for protected branches, `ci-untrusted` on ephemeral credential-free hosts. One credential per purpose, agents on workload identity. A shared wrapper enforcing project prefix, `output_uri`, tags, and image digest. Scheduled retention, a weekly rebuild check, and a published platform report. Then say plainly that the open-source server has no RBAC, so isolation is queues, credentials, and buckets rather than permissions.

02 Why is enqueue permission a security decision?

Because a task carries a repository, commit, diff, package list, and entry point, and an agent runs it. Enqueue is remote code execution on your fleet, inheriting whatever ambient cloud credentials that agent holds. So queues become the security boundary, each pool gets its own credentials, and fork pull requests get an isolated ephemeral pool with nothing attached.

03 What is the risk in ClearML storing your uncommitted diff?

It is a good feature, since a dirty working tree is still reproducible. It also means anything present but untracked at run time, like a `.env` or a service-account JSON, is captured into the task record, visible to everyone with workspace access, and persisted in backups. Remediation is deleting the task and rotating the credential anyway, so prevention is gitignore discipline plus a pre-run check.

04 Which ClearML component breaks first as you grow, and why?

Elasticsearch, on reporting rate. Metrics and console logs grow monotonically and never shrink; crossing the high disk watermark marks indices read-only, and the symptom is that metric reporting silently stops while training carries on. Second is MongoDB on task count, which shows up as a slow experiments table. The fixes are per-epoch reporting, retention, and capacity, in that order, because the first is free.

05 What does a complete ClearML backup contain?

Three parts: a `mongodump` of the `backend` and `auth` databases, an Elasticsearch snapshot, and the artifact bucket with versioning enabled. Mongo alone restores every task, model, and parameter with empty Scalars tabs, because scalars, plots, and console output live in Elasticsearch. The backup is only real once you have restored it and verified login, a known task's scalars, an artifact download, and an agent claiming a task.



pin it. Base image by digest rather than tag, a hash-pinned lock file, the seed as a connected parameter, determinism flags recorded, and the dataset consumed by version with an alias. Then a scheduled job that clones a past release, reruns it, and compares the metric within a tolerance, because otherwise reproducibility degrades silently and you find out during an incident.

07

A customer asks you to delete their data. Walk me through it.

First enumerate: dataset chunks, every child version inheriting them, task artifacts, debug samples, console logs in Elasticsearch, and backups. Then the honest part: trained weights retain the influence and for some model classes information is extractable, so if the regime treats a model as personal data the only real remedy is retraining. Which is why the architecture matters more than the runbook: version derived or pseudonymised data, keep the identity mapping in a system built for row deletes, and crypto-shred if raw data must be versioned.

08

How do you control GPU cost?

Measure first: GPU-hours consumed versus hours that produced a completed run. The waste is usually abandoned interactive sessions, runs that fail late, and sweeps that never prune. So: session timeouts and a weekly report naming owners, fail-fast validation of config and data access in the first minute, verified HPO objectives with iteration caps and `'save_top_k_tasks_only'`, and an autoscaler with a scale-to-zero floor. Spot only if the job checkpoints and the pipeline retries.

09

How do you handle multi-tenancy on the open-source server?

By being honest that it has one workspace and no RBAC. Isolation comes from queues with per-pool credentials, bucket prefixes enforced by cloud IAM, nested projects and mandatory tags by convention, and worker counts as quota. A shared wrapper makes those defaults rather than documentation. If genuine hard isolation is required (different legal entities, different data classifications) it is the paid tier or a server per tenant, and pretending otherwise fails a security review.

10

Why must a pipeline controller not run on a GPU queue?

A controller mostly sleeps while its steps execute. On a GPU worker it occupies that GPU for the pipeline's whole duration doing nothing, while its own GPU steps queue behind it, which



11 What is the failure mode of pipeline caching?

It keys on the base task, the parameters, and the code, not on the contents of input files. A step reading a mutable path has identical parameters when the data changes, so it is skipped, downstream steps consume a stale artifact, and the pipeline is green. The fix is to consume a versioned dataset, which turns a data change into a parameter change and therefore a cache miss.

12 How does a promotion become a deployment without a deploy step?

The serving endpoint is registered with `model auto-update` against a project, name, and tag rather than a model id, so moving the `production` tag is the deploy. That makes the tag vocabulary load-bearing: `prod` versus `production` means the query finds nothing and the old model silently stays. Fix four tags, set them only from the promotion task, and publish the winner so it cannot be swapped.

13 What monitoring does a ClearML platform need, in priority order?

Elasticsearch disk above 80%, because read-only is silent. Services-agent liveness, because everything automatic lives there and a dead agent presents as "the nightly did not run" with no error. Queue depth sustained over an hour, meaning a dead worker or a runaway sweep. Then Mongo disk, API 5xx rate, the rebuild-check result, and weekly GPU utilisation. Task failures are deliberately not on that list. Those belong to their authors.

14 You inherit a ClearML setup. What do you check first?

Whether fork pull requests can enqueue anywhere, and whether backups have ever been restored. Those two are an RCE path and an unrecoverable data loss. Then artifacts on the file server instead of S3, an unpinned server image, missing Elastic disk alerts, no retention job, base images by tag rather than digest, and whether promotion is a task or a click.

15 How would you audit a deployed model?

One command, not a conversation: from the model id, resolve the producing task, then report repository, commit, whether the tree was dirty, the container digest, the parameters as executed, the dataset ids from the parameters, the model metadata and tags, and the



16 **ClearML versus MLflow plus Airflow plus DVC?**

ClearML puts tracking, data versioning, and execution in one system, so a recorded run is re-executable and a promotion can deploy itself. The cost is a stateful platform: MongoDB, Elasticsearch, Redis, backups, and upgrades that run migrations. If you already run Airflow and a tracking server and your data versioning is settled, adopting ClearML replaces working components rather than filling a gap. It earns its place when you need the spine, and it is real operational cost when you do not.

17 **How do you keep researchers productive without giving everyone everything?**

A thin wrapper: they call one function and get the right project prefix, output_uri, tags, package pinning, and image digest. They enqueue to their team's queue, which is credentialed for data read and artifact write only. Promotion and serving live on queues only the platform team can reach. The result is that the safe path is also the shortest one, which is the only governance that survives contact with a deadline.

18 **What is your incident playbook for a lost server?**

Agents keep running and buffer, so there is some slack. Stand up the same pinned image, not `latest`, because a version change means a migration on top of an incident. Restore Mongo, then Elasticsearch. Verify login, a known task's scalars, an artifact download, and an agent claiming a queued task, in that order. Then reconcile tasks that finished during the outage and still show as running. The number that matters is the RTO you measured last quarter, not the one you assume.

19 **Where would you not use ClearML?**

Point-in-time feature serving with online/offline parity. That is a feature store. SQL analytics over versioned tables belongs in a table format or a warehouse. Company-wide orchestration beyond ML belongs in Airflow, Dagster, or Temporal. High-scale low-latency serving belongs in KServe or Triton directly. Fine-grained RBAC on the open-source tier. Stretching ClearML across those costs more than the second tool would.

20 **What is the single idea that connects everything?**

A recorded run that can be executed again is worth more than a log of one that cannot. Agents, queues, the registry, image digests, lock files, and backups all exist to keep that



Final self-test

- Give the thirty-second answer, then the five components and the three data stores.
- State the rules that never change: init order, `connect` 's direction, title/series, `output_uri` , `finalize` .
- Distinguish `init` , `create` , and `clone` , and explain reuse versus continuation.
- Explain why enqueue permission is code execution, and describe the fork-PR danger precisely.
- Name the three properties that make ClearML's access model sharper than it looks.
- Say which component breaks first at scale, what the symptom is, and the fix order.
- List the three parts of a backup and what a Mongo-only restore looks like.
- Name every layer that must be pinned for an eighteen-month rebuild, and how you verify it.
- State what pipeline caching keys on, and the stale-result failure it causes.
- Walk the promotion chain and explain how a tag move becomes a deployment.
- Enumerate everything an erasure request touches, and the one thing you cannot erase.
- Give the seven platform alerts in priority order, and say why task failures are not among them.
- Name three places ClearML is the wrong tool, and what you would use instead.



PREVIOUS

Senior · Detailed

NEXT UP

Senior · Tips & Tricks





MLOps MENA
COMMUNITY

العربية العربية



[Join the Community ↗](#)



MLOps MENA
COMMUNITY

Free MLOps and AI learning for engineers across the Middle East and North Africa — live sessions, three structured roadmaps, open cohort courses, study groups, and mentorship. We also train, staff, and build for companies across the region.



LEARN

[Roadmaps](#)

[Student Guides](#)

[Courses](#)

[The MLOps Practitioner](#)

[Sessions](#)

[Articles](#)

COMMUNITY

[Meet the team](#)

[Services for companies](#)

[FAQ](#)

[Contact us](#)

[Privacy Policy](#)

WHERE WE ARE

[WhatsApp](#)

[Discord](#)

[LinkedIn](#)

[X](#)

[YouTube](#)

[GitHub](#)



MLOps MENA
COMMUNITY

العربية



Join the Community ↗



Learning space

Free to join this week with code **FREECORETEAM**

© 2026 MLOps MENA Community. Free learning for the community.

Built by the community, for the community.